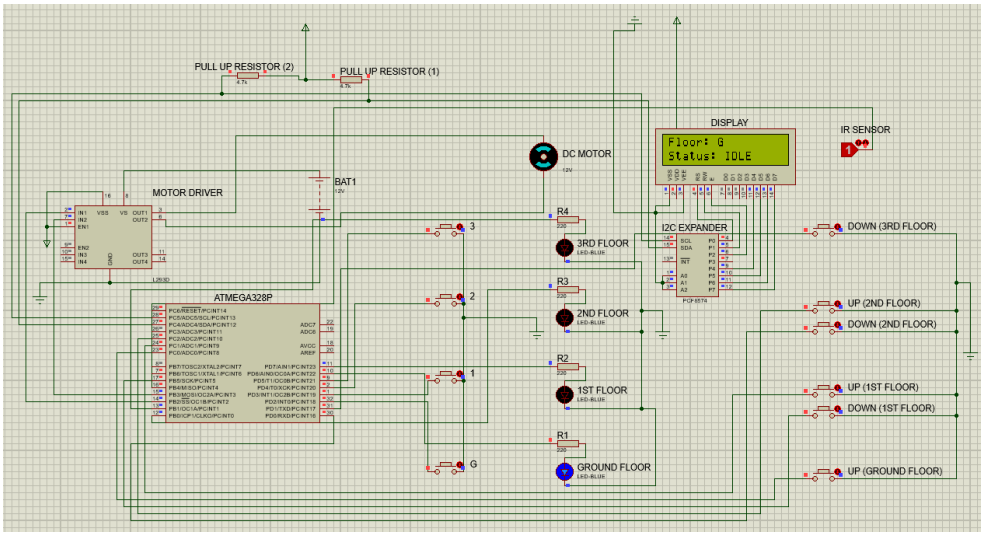
Project Report: 4-Floor Elevator Control System Simulation

1. Design Approach:

Since I use an elevator about 10 times a day, I have a good idea of how it works in real life. When designing this project, I first pictured the daily experience: walking up to the lift, pressing the up or down arrow, and waiting for the doors to open. I also immediately thought of safety—the doors shouldn't close if someone is standing in the way!

Once inside, you press the button for your destination, and the lift takes you there. But what makes elevators really fascinating to me is their smart algorithm. For example, if I am going to the 3rd floor, and someone on the 2nd floor presses the button while I am on my way up, the lift is smart enough to stop and pick them up. I wanted to design my simulation with this exact same brilliant logic. In real life, an elevator is driven by a powerful traction motor and a system of steel cables to hoist the cabin. To realistically represent this physical movement in my simulation, I included a bidirectional DC motor to show the lift's core mechanical drive system in action.

Technically, my goal was to write code that doesn't freeze when multiple people press buttons at the same time. I built the simulation in Proteus using an ATmega328P microcontroller. Because elevators have so many buttons, I was running out of pins. To solve this, I used a PCF8574 I2C expander to connect my LCD screen using only two pins, and an L293D motor driver to control the DC motor's direction.



2.Implementation Process:

I broke the project down into hardware wiring and software logic:

- **Buttons & Lights:** I set up 4 buttons for inside the cabin, and 6 "UP/DOWN" buttons for the outside halls. To make it exactly like a real building, I did not include an "UP" button on the top floor or a "DOWN" button on the ground floor. I also added 4 LEDs to show which floor the lift is currently on.
- **Code Logic:** The brain of the lift uses three states: IDLE, UP, and DOWN. To make sure the lift never misses a button press, I wrote a custom smartDelay() function. This lets the lift pause to open its doors, but secretly keeps reading the buttons in the background.
- **LCD Screen:** Getting the I2C LCD to work in Proteus was tricky. As I was running out of pins, I used a PCF8574 I2C expander to connect my LCD screen using only two pins to the ATmega328P microcontroller.
- **Door Safety Sensor:** I used a freed-up pin (Pin 12) to create an IR Obstacle Sensor and in the place of this sensor, the component that I used in the Proteus is “LOGICSTATE” which works like an IR Sensor. In the code, when the doors are trying to close, it checks this sensor. If someone is blocking the door, the code gets trapped in a loop and shows "DOOR BLOCKED!" on the screen until they move out of the way.

3. Results:

The final simulation works really well and does exactly what I planned:

- **Smart Routing:** The lift remembers multiple button presses. It finishes all the pickups in one direction before turning around, just like a real elevator.
- **User Interface:** The LCD screen updates instantly to show the floor number and what the doors are doing.
- **Safety:** The simulated door sensor works perfectly. If the sensor is triggered, the motor refuses to run and the doors stay open along with a notification on the display showing “DOOR BLOCKED!”.